Agent Engineering London / October 2026

Software Development as Constrained Optimization

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**, Founder & CEO of **Pydantic Labs**

Open source

FREE – WHAT YOU BUILD WITH

Pydantic Validation
Contracts for data, 1 billion downloads a month

Pydantic AI
Production grade agent framework

Pydantic Monty
Python sandbox for code written by agents, starts in 1ms

Pydantic Logfire

COMMERCIAL – WHERE YOU RUN IT

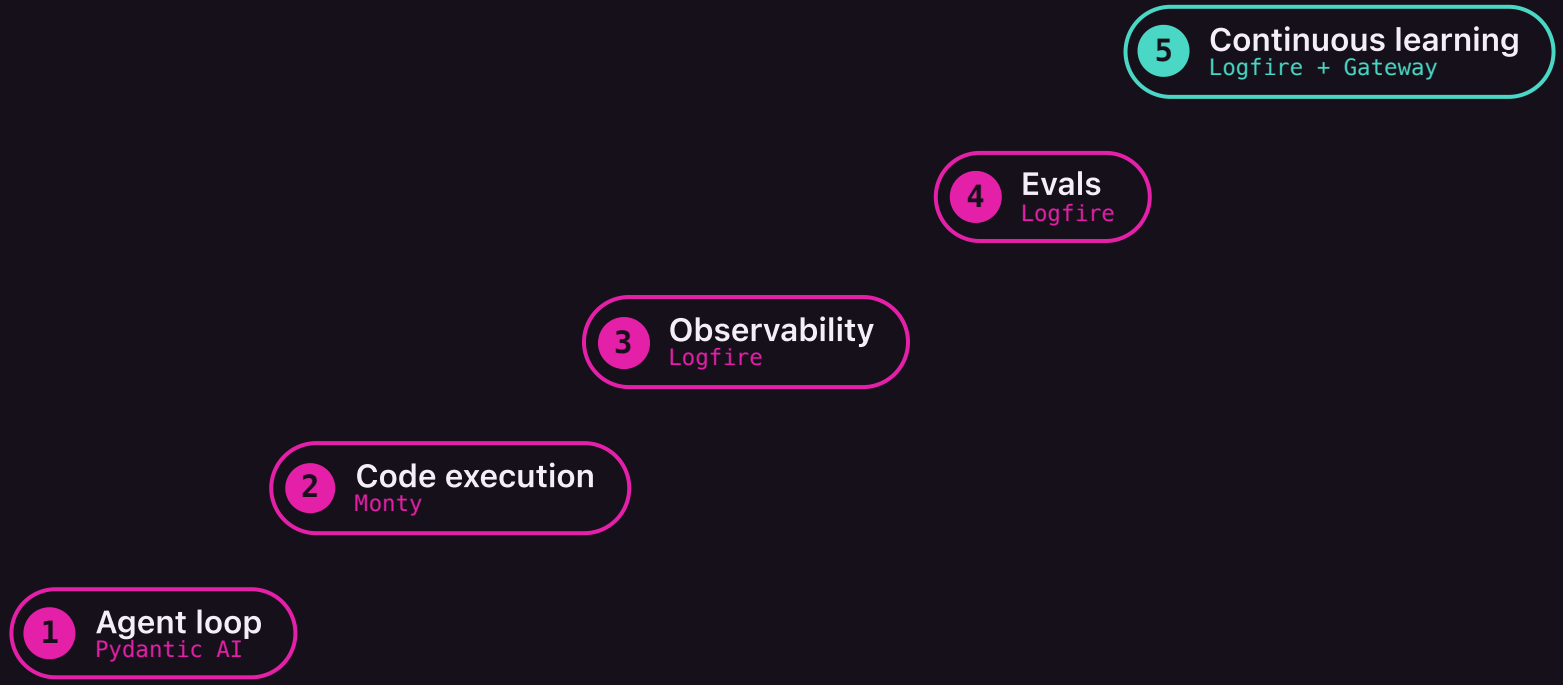
AI Observability
Traces, evals and prompt management for agents

Infra Observability
Logs, metrics and traces for the whole application

AI Gateway
One endpoint, every model

Sandboxes soon
Hosted code execution, built on Monty

What Pydantic does, in five steps



That's the day job. This talk is about something else.

How I think about building software now.

It's a theory. I'm not certain it's right today, let alone tomorrow.

One fairly simple idea, some dodgy AI-slop metaphors so everyone gets it, then three real projects for colour.

Software development is a constrained, non-convex optimization problem.

One sentence, three parts

Optimization

There are metrics and you want them better: speed, compatibility, memory, signups.

The agent is the optimizer. Stochastic, tireless, not picky about how.

Constrained

Some things must stay true whatever the metric says: the spec, security, no `unsafe`.

Gates, not weights. The optimizer never gets to trade them.

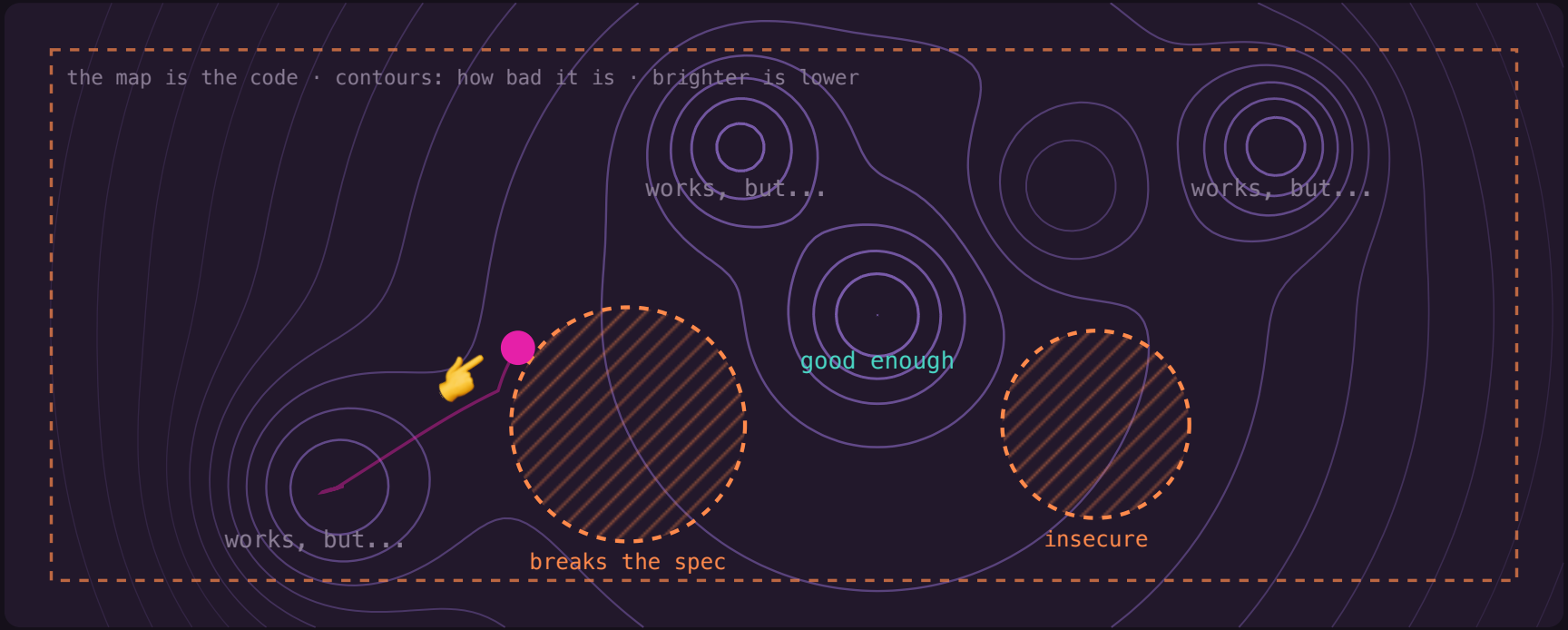
Non-convex

There is no single downhill path. Plenty of places "work" but aren't good, and the optimizer will stop in one.

Someone has to poke it.

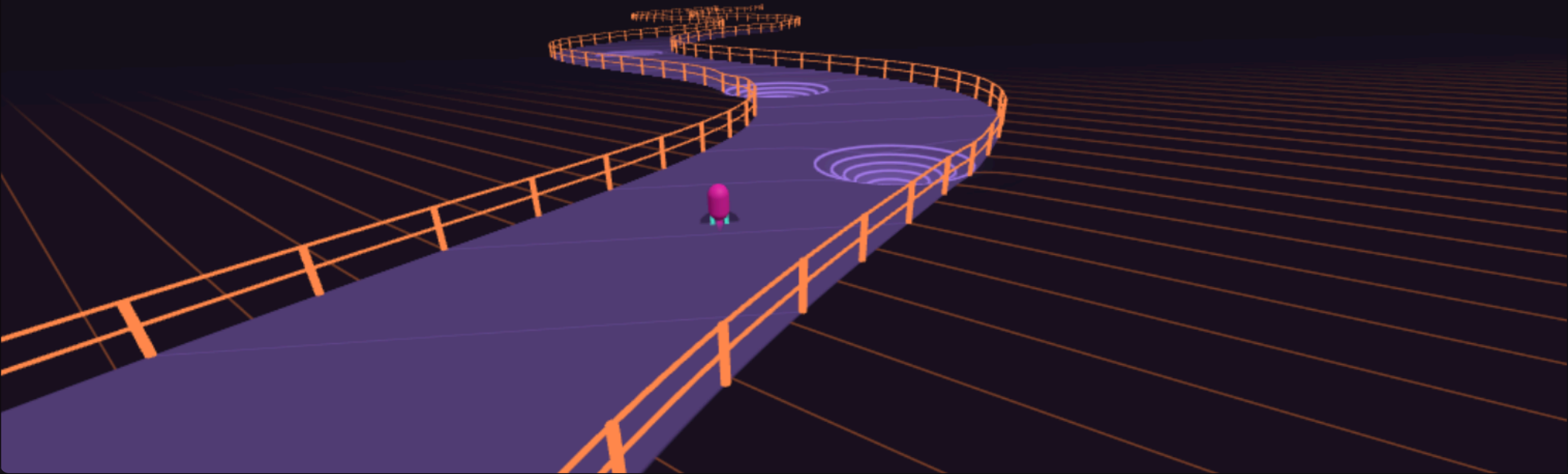
My job moved from finding **x** to choosing **f** and **g**. Review is one of the **g**'s.

The bowl, from above



Yes, the bowl is a convex *function*. No, I don't know why mathematicians called it that either. The dips make it non-convex, and the dips are where the agent stops. The hatched areas are the constraints: the ball never goes in, however good the slope looks.

But we never reach Perfect, the descent is indefinite



jiter: the obvious one

Constraints how we enforce them

JSON compliance · agree with serde_json on every document in the json-cases corpus: ~3,300 files incl. JSONTestSuite, invalid cases swept byte by byte, nesting to 100,000 deep

Known differences are written down · three exceptions in known_difference(); anything else fails the test, and a missing corpus fails too

Nothing undefined · differential fuzzing against serde_json, 300 s per target in CI; cargo careful for UB; recursion capped at 200, digits at 4,300

The order of work · the corpus landed on 17 August 2026, the differential test on the 20th, and only then did optimization resume

Metrics how the model measures them

Speed, on every PR · CodSpeed with ASLR off, Cargo.lock committed and the corpus ref pinned, so the number is the number

Speed, per shape · criterion benches paired with serde_json for strings, ints, floats, deep nesting and the error path

Allocations · a counting GlobalAlloc example: deterministic, so it can be trusted on a busy machine where timings can't

Python · bench against orjson, ujson and the stdlib, with the string cache on and off

4.1x faster than serde_json over the whole corpus · 26 commits since the tests landed, most of them co-authored with Claude

Monty: the big one

Constraints how we enforce them

No unsafe · unsafe_code = "deny" across the workspace, two documented expect()s; the agent rule is "NEVER write unsafe code, leave a todo!()"

No escape, by construction · the capability does not exist in the bytecode VM: no FFI, no C dependencies, no filesystem or network unless granted

Resource limits that hold · the hard memory limit lives in the allocator; ~80 tests in resource_limits.rs; compile-fail tests; miri

Every exploit becomes a wall · Hack Monty round 3: a read-write mount overlapping a read-only one let the weaker mode win; now overlapping mounts are rejected, with a regression test

Metrics how the model measures them

CPython compatibility · 657 test files, ~17k asserts, run against Monty and embedded CPython; CPython fills in the expected values; "NEVER MARK TESTS AS XFAIL"

Hard errors, not drift · unsupported syntax is NotImplementedError at parse time; 40 pages in limitations/ that every PR must keep in sync

Speed · CodSpeed on every PR, criterion pairs against CPython; a new sandbox in 0.8 ms

Memory and size · ~2 MB per worker; verbosity is reviewed, not measured

~207k lines of Rust in 18 crates · about a third of commits since November 2025 carry a Claude trailer

Constraints we couldn't afford before

The hosted Monty relay shipped last week with a `verification/` directory written alongside the code:

- **11 Dafny and 5 TLA+ models**: admission, lease replacement, dispatch, broker lifetime, recovery
- **13 Kani harnesses** that import the production decision functions, over the full input domain
- **86 implementation mutations** that must fail, so an always-deny implementation cannot pass
- An **eight-job CI matrix**, required whenever relay code changes; each proof receipt is bound to the hash of its inputs
- A README that says what it does **not** prove: the compiler, the scheduler, the deployed binary

```
predicate Safe(s: State, valid: set<int>,
               authLimit: nat, lookupLimit: nat,
               sessionLimit: nat) {
  s.lookups <= s.pending &&
  s.pending * s.authorized == {} &&
  s.sessions <= s.authorized &&
  s.authorized <= valid &&
  |s.pending| <= authLimit &&
  |s.lookups| <= lookupLimit &&
  |s.sessions| <= sessionLimit
}
```

A year ago nobody would have paid for a Dafny proof of a WebSocket relay. When the agent writes it, the only question is whether the fence is worth having.

Logfire: the least obvious one

Constraints how we enforce them

Tenant isolation in the types · every scan carries a ProjectId, never an Option<ProjectId>, so a caller without a project fails to compile; a missing scope rejects the query, never continues

The gate is mergify, not a prompt · 70 pre-commit hooks incl. 18 house lint rules, pyright strict, a 25-job check, 100% coverage on touched packages, old app booted on the new schema, one human approval

Proofs where tests can't reach · Dafny for the DB kernel, TLA+ for cache eviction, checked in CI

Incidents become rules · a project filter was dead code for 308 days; now project_id is non-optional and a ratchet test only lets the list of tenancy gaps shrink. MUST rules each cite a real example

Metrics how the model measures them

Query speed, measured not gated · paired A/B benchmark on every PR posts the latency diff and always exits 0; 408 corpus queries run against staging every five minutes

Logfire on Logfire · every environment ships its telemetry to a meta project; SLOs live in code

Ease of use · a feature "is not done without a working walkthrough"; 30 review agents, all advisory

The loop itself · a weekly agent proposes one CI speed-up with a measured wall-clock gain

~1.25M lines of Python, ~460k of Rust · 48 contributors · at least 40% of commits since April carry a Claude trailer

What I'd do tomorrow

- **One objective.** Pick the metric. Turn every other thing you care about into a constraint with a threshold: "latency under 200 ms", not "also make it fast".
- **Gates, not weights.** If a violation is unacceptable it fails the build. Security is never a score.
- **Every incident becomes a permanent, executable constraint.** That is the job: given a bad solution, produce the cheap check it violates.
- **Make the loop fast and deterministic.** The verifier is your leverage; flaky or slow CI caps how hard you can push.

Where the metaphor breaks

- **Constraints are sampled, not proven.** CI checks points, not the region. Types and proofs are the exception, which is why the Dafny slide matters.
- **The optimizer can read the constraints.** A solver can't lobby you to relax a bound; an agent will edit the test. Fences need tamper-protection: branch protection, "never xfail", a human approval, credentials it doesn't have.
- **The real objective is a proxy.** Signups, revenue, "ease of use" aren't measurable until deployed. Goodhart's law applies to you, not just the model.

Why I'm optimistic

Defining and maintaining the fences on an endless slope is a harder, more complex and more valuable job than typing out the code ever was.

Not a software factory. I still write code, and my review is often the most important constraint. But the person who formulates the problem is worth more today than the person who wrote the code was a year ago.

Thank you



pydantic.dev/contact

`@samuelcolvin` · `@pydantic`